

Advanced Computing Project (2025/26)

Storage system for optimizing LLM checkpointing in HPC

André Miranda
MEI, PG60238

Gustavo Barros
MEI, PG61527

José Soares
MEI, PG60277

Miguel Carvalho
MEI, PG61153

Abstract—Training Large Language Models (LLMs) in High-Performance Computing (HPC) environments is a time-consuming process that requires frequent checkpointing to ensure fault tolerance. However, concurrent writes of massive model states often saturate the shared Parallel File System (PFS), leading to significant I/O contention and training stalls. This paper proposes a policy-driven, two-tier storage orchestration system designed to mitigate the “noisy neighbor” problem in HPC clusters. By transparently redirecting checkpoints to node-local SSDs and employing a centralized orchestrator to manage asynchronous migration to the PFS via a Token Bucket algorithm, we enable granular Quality of Service (QoS) control. Our evaluation on the Deucalion supercomputer demonstrates that

...
Index Terms—High-Performance Computing, Deucalion, A64FX, AMD EPYC 7742, NVIDIA A100, Memory Hierarchy, Large Language Models, Checkpointing, Parallel File Systems, I/O Contention, Quality of Service (QoS), Distributed Systems, Resource Management.

I. INTRODUCTION

The rapid growth of Large Language Models (LLMs) has significantly increased the computational and storage demands of modern High-Performance Computing (HPC) systems. Training such models requires long-running jobs over distributed infrastructures, where fault tolerance becomes essential. Checkpointing—periodically saving the model state—is the standard mechanism to ensure progress is not lost due to failures. However, as model sizes scale to billions of parameters, checkpoint operations generate massive I/O workloads.

In shared HPC environments, these workloads are typically directed to a Parallel File System (PFS), which is simultaneously accessed by multiple users and applications. This leads to contention, particularly when several jobs attempt to write checkpoints concurrently. The resulting “noisy neighbor” effect can degrade performance cluster-wide, increasing latency and even stalling training processes.

Existing approaches either optimize checkpoint size or leverage faster intermediate storage, but often lack coordination mechanisms to regulate global I/O pressure. This paper proposes a policy-driven, two-tier storage orchestration system that combines node-local SSD staging with centralized rate control. By decoupling checkpoint generation from persistent storage and introducing a Token Bucket-based migration strategy, the system provides predictable Quality of Service (QoS) while maintaining transparency to user applications.

We evaluate the proposed system on the Deucalion supercomputer, demonstrating its ability to reduce I/O contention

and improve overall system stability during large-scale LLM training.

II. BACKGROUND AND MOTIVATION

A. Checkpointing in HPC

Checkpointing is a fundamental technique for fault tolerance in HPC applications. In distributed deep learning, checkpoints typically include model weights, optimizer states, and training metadata. As model sizes increase, checkpoint sizes can reach tens or hundreds of gigabytes, making frequent persistence costly.

B. Parallel File Systems and Contention

Parallel File Systems such as Lustre are designed for high throughput, but their performance degrades under bursty and uncoordinated workloads. Concurrent checkpointing from multiple jobs creates synchronized I/O spikes, overwhelming metadata servers and storage targets.

C. Motivation

The main motivation of this work is that checkpointing is not latency-critical for persistence, but it is latency-critical for training continuation. This distinction enables decoupling the checkpoint write path into fast local operations and controlled background transfers.

III. SYSTEM DESIGN

The proposed system adopts a two-tier storage architecture combined with a centralized control plane.

A. Design Overview

The system consists of three main components:

- **Interception Layer:** Transparently redirects checkpoint writes.
- **Local Storage Tier:** Node-local SSDs for fast staging.
- **Control Plane:** Central orchestrator managing global I/O behavior.

B. I/O Interception Mechanism

The system intercepts POSIX write calls associated with checkpoint operations and redirects them to local SSD storage.

C. Local Storage Tier

Each compute node uses its local SSD as a staging area. This eliminates immediate contention on the PFS and enables high-throughput writes.

D. Global Orchestrator

The orchestrator maintains system-wide state, including:

- Active jobs and checkpoint queues
- Current PFS load
- Token allocation per job

IV. QoS-AWARE DATA MIGRATION

A. Token Bucket Model

To regulate PFS access, each job is assigned a token bucket defining its allowed write rate. Tokens accumulate over time and are consumed during data transfers.

B. Scheduling Policy

The orchestrator schedules checkpoint flushes based on:

- Token availability
- Queue size
- System load

C. Fairness and Isolation

The Token Bucket mechanism ensures that no job exceeds its allocated bandwidth, providing fairness and mitigating interference.

V. IMPLEMENTATION

A. Prototype Design

The system is implemented as a user-space middleware layer, avoiding kernel modifications. The interception layer is realized using dynamic linking techniques.

B. Data Management

Checkpoint files are stored locally with metadata tracking their migration status. A background daemon handles transfers to the PFS.

C. Fault Tolerance

To ensure reliability, the system maintains redundancy metadata and guarantees that checkpoints are eventually persisted to the PFS.

D. Integration with HPC Workflows

The system is compatible with existing job schedulers and deep learning frameworks, requiring no changes to user applications.

VI. BENCHMARKING

Experiments were conducted on the Deucalion supercomputer, featuring heterogeneous compute nodes and a shared PFS. Workloads simulate LLM checkpointing behavior with varying sizes and frequencies.

A. Workloads

We evaluate the system using synthetic and realistic checkpointing workloads designed to mimic Large Language Model (LLM) training behavior:

- TODO

B. Metrics

- Checkpoint latency
- Training stall duration
- PFS throughput and variability
- Fairness index across jobs
- I/O congestion
- CPU/GPU and memory usage

VII. RELATED WORK

The optimization of checkpointing in High-Performance Computing (HPC) has been extensively studied, primarily focusing on reducing write latency and mitigating Parallel File System (PFS) congestion.

A. Architectural and Tiering Approaches

Several systems utilize hierarchical storage to hide I/O latency. **Check-N-Run** [5] focuses on staging checkpoints in local node resources (DRAM or SSDs), allowing training to resume immediately while data is moved to the PFS in the background. However, it lacks a global coordination mechanism to prevent PFS saturation during the background flush. **DeepFreeze** [6] (similar to Monarch) employs transparent storage tiering to accelerate the write path by abstracting complex local tiers like NVMe or PMEM from the user. While effective at the node level, these systems often ignore the "noisy neighbor" effect in shared environments.

B. Adaptive and Optimization Techniques

Other research focuses on the frequency and size of checkpoints. **CheckFreq** [4] introduces an algorithmic approach to identify optimal checkpoint boundaries and pipelines the process with training iterations, dynamically adjusting frequency to maintain overhead below a threshold. **BitSnap** [7] addresses the data volume by employing sparsification and quantization, reducing checkpoint sizes from 16-bit to 4-bit weights before they reach the I/O layer. While these reduce total I/O pressure, they do not manage the storage hierarchy or provide holistic Quality of Service (QoS) guarantees.

C. Software-Defined Storage and QoS Control

Our work is heavily informed by **PADLL** [8], a storage middleware that enables application-agnostic QoS control for both data and metadata workflows. PADLL utilizes a decoupled Software-Defined Storage (SDS) architecture, where data plane stages intercept POSIX calls and a centralized control plane coordinates I/O rates across all jobs. While PADLL is a general-purpose QoS framework, our system specializes this paradigm for the specific lifecycle of LLM checkpoints—integrating node-local SSD staging with PADLL-inspired Token Bucket rate limiting to ensure cluster harmony during massive model state migrations.

VIII. RESULTS

Colocar resultados em geral e em específico para cada métrica?

IX. DISCUSSION

A. Trade-offs

B. Scalability

C. Limitations

X. CONCLUSIONS

TODO

XI. FUTURE WORK

Future work will focus on...

TODO

REFERENCES

- [1] EuroHPC Joint Undertaking, “Access Calls – Technical Guidelines, Version 6 (26.05.2025),” 2025. Link
- [2] Fujitsu Ltd., “FUJITSU Processor A64FX – Architecture and Specifications,” 2020. Link
- [3] AMD, “AMD EPYC 7742 Processor (Rome) – Product Specifications and Performance Guide,” 2019–2024. Link
- [4] J. Mohan, A. Phanishayee, and V. Chidambaram, “CheckFreq: Frequent, Fine-Grained DNN Checkpointing,” in *19th USENIX Conference on File and Storage Technologies (FAST 21)*, pp. 203–216, Feb. 2021. <https://www.usenix.org/conference/fast21/presentation/mohan>
- [5] A. Eisenman et al., “Check-N-Run: a Checkpointing System for Training Deep Learning Recommendation Models,” in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pp. 929–943, Apr. 2022. <https://www.usenix.org/conference/nsdi22/presentation/eisenman>
- [6] B. Nicolae, J. Li, J. Wozniak, G. Bosilca, M. Dorier, and F. Cappello, “DeepFreeze: Towards Scalable Asynchronous Checkpointing of Deep Learning Models,” in *CCGrid 2020*, pp. 172–181, May 2020. DOI: 10.1109/CCGrid49817.2020.00-76
- [7] Y. Peng et al., “BitSnap: Checkpoint Sparsification and Quantization in LLM Training,” *arXiv preprint arXiv:2511.12376*, 2025. <https://arxiv.org/abs/2511.12376>
- [8] R. Macedo et al., “PADLL: Taming Metadata-intensive HPC Jobs Through Dynamic, Application-agnostic QoS Control,” *arXiv preprint arXiv:2302.06418*, 2023. <https://arxiv.org/abs/2302.06418>